

COMP9319 Web Data Compression and Search

BWT, MTF and Pattern Matching

1

BWT

- Burrows–Wheeler transform (BWT) is an algorithm used to prepare data for use with data compression techniques such as bzip2.
- It was invented by Michael Burrows and David Wheeler in 1994 at DEC SRC, Palo Alto, California.
- It is based on a previously unpublished transformation discovered by Wheeler in 1983.

2

A simple example

Input:
#BANANAS

3

All rotations

#BANANAS
S#BANANA
AS#BANAN
NAS#BANA
ANAS#BAN
NANAS#BA
ANANAS#B
BANANAS#

4

Sort the rows

#BANANAS
ANANAS#B
ANAS#BAN
AS#BANAN
BANANAS#
NANAS#BA
NAS#BANA
S#BANANA

5

Output

#BANANAS
ANANAS#B
ANAS#BAN
AS#BANAN
BANANAS#
NANAS#BA
NAS#BANA
S#BANANA

6

Now the inverse, for decoding...

Input:

S
B
N
N

A
A
A

7

7

First add

S
B
N
N

A
A
A

8

8

Then sort

A
A
A
B
N
N
S

9

9

Add again

S#
BA
NA
NA
#B
AN
AN
AS

10

10

Then sort

#B
AN
AN
AS
BA
NA
NA
S#

11

11

Then add

S#B
BAN
NAN
NAS
#BA
ANA
ANA
AS#

12

12

Then sort

#BA
ANA
ANA
AS#
BAN
NAN
NAS
S#B

13

13

Then add

S#BA
BANA
NANA
NAS#
#BAN
ANAN
ANAS
AS#B

14

14

Then sort

#BAN
ANAN
ANAS
AS#B
BANA
NANA
NAS#
S#BA

15

15

Then add

S#BAN
BANAN
NANAS
NAS#B
#BANA
ANANA
ANAS#
AS#BA

16

16

Then sort

#BANA
ANANA
ANAS#
AS#BA
BANAN
NANAS
NAS#B
S#BAN

17

17

Then add

S#BANA
BANANA
NANAS#
NAS#BA
#BANAN
ANANAS
ANAS#B
AS#BAN

18

18

Then sort

#BANAN
ANANAS
ANAS#B
AS#BAN
BANANA
NANAS#
NAS#BA
S#BANA

19

19

Then add

S#BANAN
BANANAS
NANAS#B
NAS#BAN
#BANANA
ANANAS#
ANAS#BA
AS#BANA

20

20

Then sort

#BANANA
ANANAS#
ANAS#BA
AS#BANA
BANANAS
NANAS#B
NAS#BAN
S#BANAN

21

21

Then add

S#BANANA
BANANAS#
NANAS#BA
NAS#BANA
#BANANAS
ANANAS#B
ANAS#BAN
AS#BANAN

22

22

Then sort (???)

#BANANAS
ANANAS#B
ANAS#BAN
AS#BANAN
BANANAS#
NANAS#BA
NAS#BANA
S#BANANA

23

23

Implementation

Do we need to represent the table in the encoder?

No, a single pointer for each row is needed.

24

24

BWT(S)

```
function BWT (string s)
  create a table, rows are all possible
    rotations of s
  sort rows alphabetically
  return (last column of the table)
```

25

25

InverseBWT(S)

```
function inverseBWT (string s)
  create empty table

  repeat length(s) times
    insert s as a column of table before first
      column of the table // first insert creates
        first column
    sort rows of the table alphabetically
  return (row that ends with the 'EOF' character)
```

26

26

Move to Front (MTF)

Reduce entropy based on local frequency correlation

Usually used for BWT before an entropy-encoding step

Author and detail:

Original paper at cs9319/papers

http://www.arturocampos.com/ac_mtf.html

27

27

Example: abaabacad

Symbol	Code	List
a	0	abcde.....
b	1	bacde.....
a	1	abcde.....
a	0	abcde.....
b	1	bacde.....
a	1	abcde.....
c	2	cabde.....
a	1	acbde.....
d	3	dacbe.....

To transform a general file, the list has 256 ASCII symbols.

28

28

BWT compressor vs ZIP

ZIP (i.e., LZW based)			BWT+RLE+MTF+AC		
File Name	Raw Size	PKZIP Size	PKZIP Bits/Byte	BWT Size	BWT Bits/Byte
bib	111,261	35,821	2.58	29,567	2.13
book1	768,771	315,999	3.29	275,831	2.87
book2	610,856	209,061	2.74	186,592	2.44
geo	102,400	68,917	5.38	62,120	4.85
news	377,109	146,010	3.10	134,174	2.85
obj1	21,504	10,311	3.84	10,857	4.04
obj2	246,814	81,846	2.65	81,948	2.66

29

From <http://marknelson.us/1996/09/01/bwt/>

29

Other ways to reverse BWT

Consider $L = \text{BWT}(S)$ is composed of the symbols $V_0 \dots V_{N-1}$, the transformed string may be parsed to obtain:

The number of symbols in the substring $V_0 \dots V_{i-1}$ that are identical to V_i .

For each unique symbol, V_i , in L , the number of symbols that are lexicographically less than that symbol.

30

30

Example

Position	Symbol	# Matching	Symbol	# LessThan
0	B	0		
1	N	0	A	0
2	N	1	B	3
3	[	0	N	4
4	A	0	[	6
5	A	1		
6	]	0	]	7
7	A	2		

31

31

???????]

Position	Symbol	# Matching	Symbol	# LessThan
0	B	0		
1	N	0	A	0
2	N	1	B	3
3	[	0	N	4
4	A	0	[	6
5	A	1		
6	]	0	]	6
7	A	2		7

32

32

??????A]

Position	Symbol	# Matching	Symbol	# LessThan
0	B	0		
1	N	0	A	0
2	N	1	B	3
3	[	0	N	4
4	A	0	[	6
5	A	1		
6	]	0	]	7
7	A	2		

33

33

?????NA]

Position	Symbol	# Matching	Symbol	# LessThan
0	B	0		
1	N	0	A	0
2	N	1	B	3
3	[	0	N	4
4	A	0	[	6
5	A	1		
6	]	0	]	6
7	A	2		7

34

34

????ANA]

Position	Symbol	# Matching	Symbol	# LessThan
0	B	0		
1	N	0	A	0
2	N	1	B	3
3	[	0	N	4
4	A	0	[	6
5	A	1		
6	]	0	]	7
7	A	2		

35

35

???NANA]

Position	Symbol	# Matching	Symbol	# LessThan
0	B	0		
1	N	0	A	0
2	N	1	B	3
3	[	0	N	4
4	A	0	[	6
5	A	1		
6	]	0	]	6
7	A	2		7

36

36

??ANANA]

Position	Symbol	# Matching	Symbol	# LessThan
0	B	0		
1	N	0	A	0
2	N	1	B	3
3	[	0	N	4
4	A	0	[	6
5	A	1	]	7
6	]	0		
7	A	2		

37

37

?BANANA]

Position	Symbol	# Matching	Symbol	# LessThan
0	B	0		
1	N	0	A	0
2	N	1	B	3
3	[	0	N	4
4	A	0	[	6
5	A	1	]	7
6	]	0		
7	A	2		

38

38

[BANANA]

Position	Symbol	# Matching	Symbol	# LessThan
0	B	0	A	0
1	N	0	B	3
2	N	1	N	4
3	[	0	[	6
4	A	0	]	7
5	A	1		
6	]	0		
7	A	2		

39

39

[BANANA]

Position	Symbol	# Matching	Symbol	# LessThan
0	B	0	A	0
1	N	0	B	3
2	N	1	N	4
3	[	0	[	6
4	A	0	]	7
5	A	1		
6	]	0		
7	A	2		

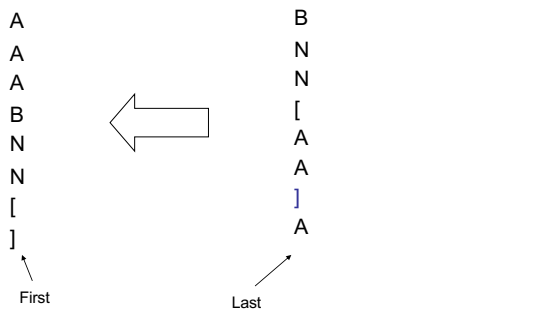
Occ / Rank

c[i]

40

40

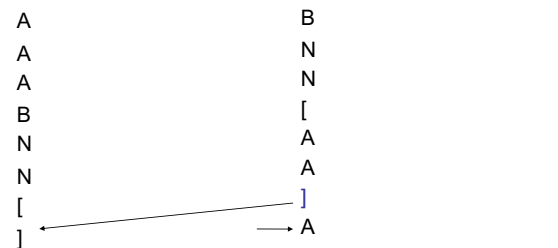
An illustration



41

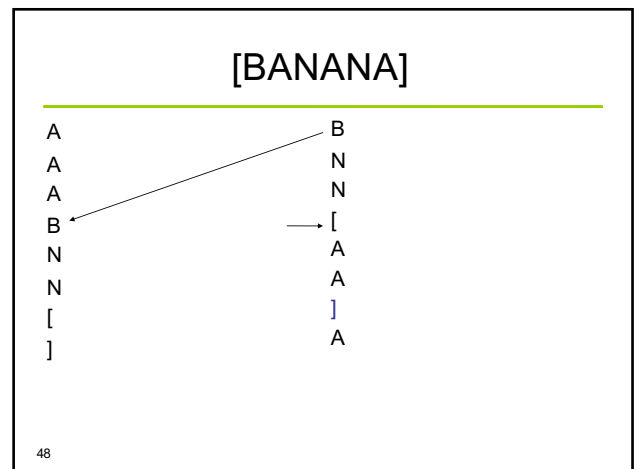
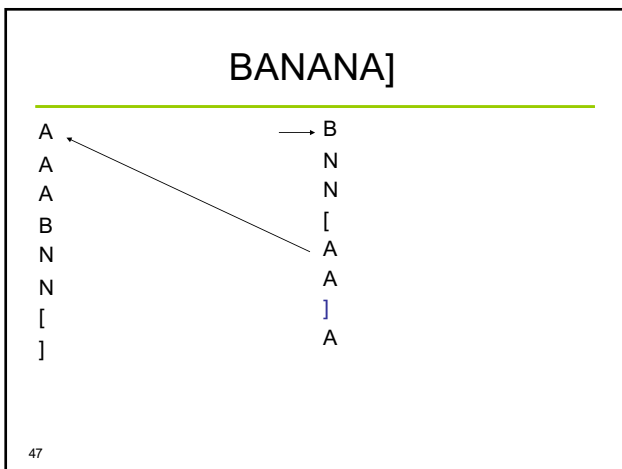
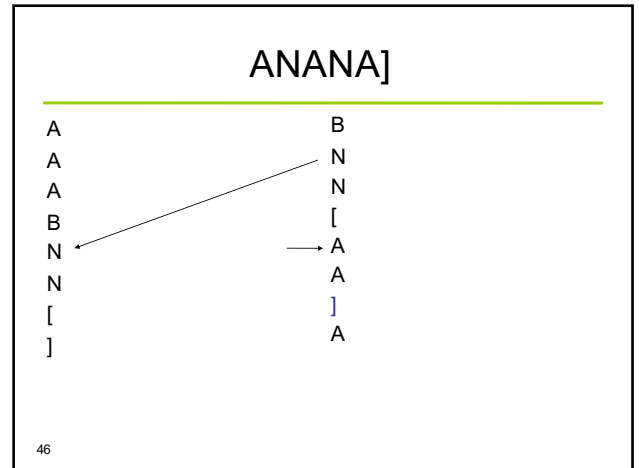
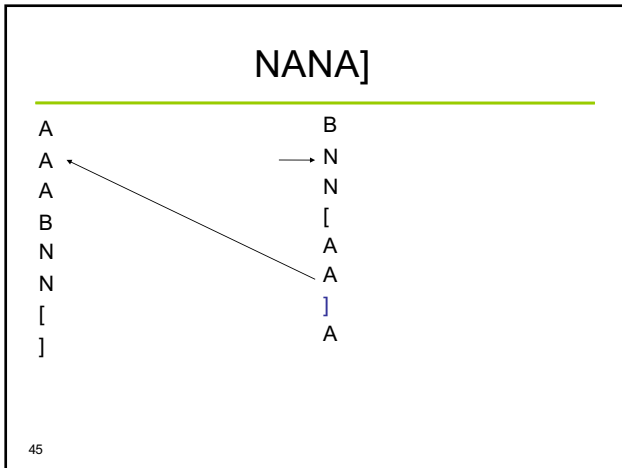
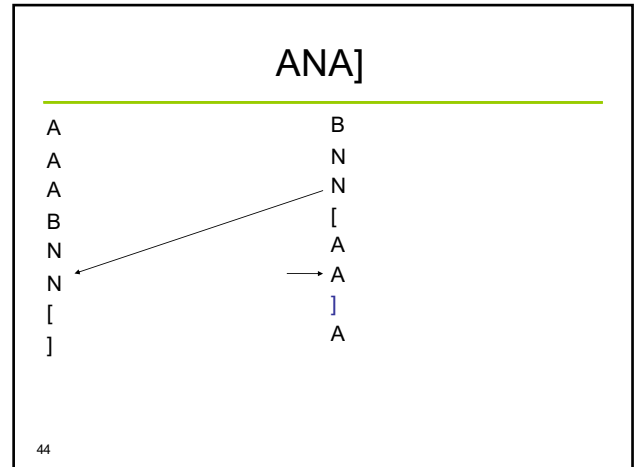
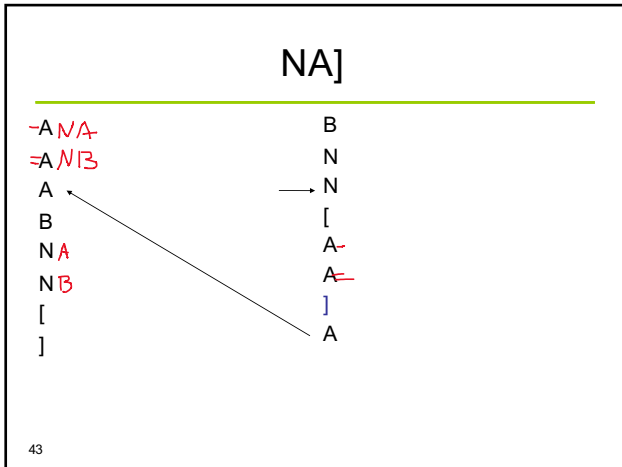
41

A]



42

42



Dynamic BWT ?

Instead of reconstructing BWT, local reordering from the original BWT.

Details:

Salson M, Lecroq T, Léonard M and Mouchard L (2009). "A Four-Stage Algorithm for Updating a Burrows–Wheeler Transform". Theoretical Computer Science 410 (43): 4350.

49

49

Search

50

50

What is Pattern Matching?

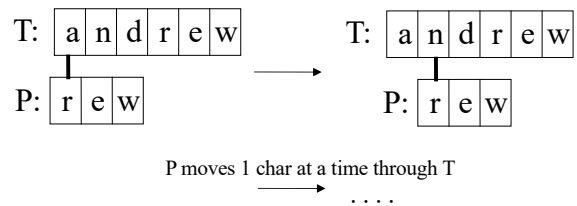
- Definition:
 - given a text string T and a pattern string P, find the pattern inside the text
 - T: "the rain in spain stays mainly on the plain"
 - P: "n th"

51

51

The Brute Force Algorithm

- Check each position in the text T to see if the pattern P starts in that position



52

52

Analysis

- Brute force pattern matching runs in time $O(mn)$ in the worst case.
- But most searches of ordinary text take $O(m+n)$, which is very quick.

53

continued

53

- The brute force algorithm is fast when the alphabet of the text is large
 - e.g. A..Z, a..z, 1..9, etc.
- It is slower when the alphabet is small
 - e.g. 0, 1 (as in binary files, image files, etc.)

54

continued

54

- Example of a worst case:
 - T: "aaaaaaaaaaaaaaaaaaaaaaah"
 - P: "aaah"
- Example of a more average case:
 - T: "a string searching example is standard"
 - P: "store"

55

55

The KMP Algorithm

- The Knuth-Morris-Pratt (KMP) algorithm looks for the pattern in the text in a *left-to-right* order (like the brute force algorithm).
- But it shifts the pattern more intelligently than the brute force algorithm.

56

continued

56

Summary

- If a mismatch occurs between the text and pattern P at P[j], what is the *most* we can shift the pattern to avoid wasteful comparisons?

57

57

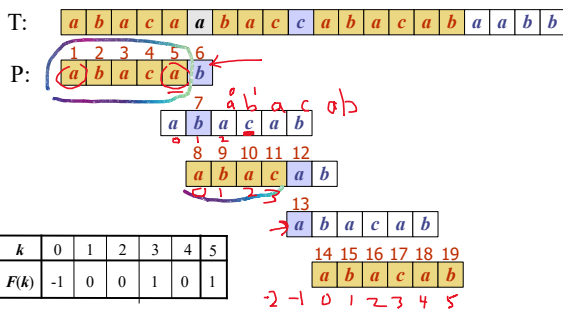
Summary

- If a mismatch occurs between the text and pattern P at P[j], what is the *most* we can shift the pattern to avoid wasteful comparisons?
- *Answer:* the largest prefix of P[0 .. j-1] that is a suffix of P[1 .. j-1]

58

58

Example



59

59

KMP Advantages

- KMP runs in optimal time: $O(m+n)$
 - very fast
- The algorithm never needs to move backwards in the input text, T
 - this makes the algorithm good for processing very large files that are read in from external devices or through a network stream

60

60

KMP Disadvantages

- KMP doesn't work so well as the size of the alphabet increases
 - more chance of a mismatch (more possible mismatches)
 - mismatches tend to occur early in the pattern, but KMP is faster when the mismatches occur later

61

61

The Boyer-Moore Algorithm

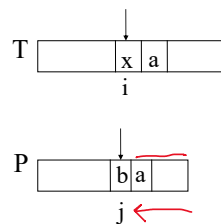
- The Boyer-Moore pattern matching algorithm is based on two techniques.
 1. The *looking-glass* technique
 - find P in T by moving *backwards* through P, starting at its end

62

62

- 2. The *character-jump* technique
 - when a mismatch occurs at $T[i] \neq x$
 - the character in pattern $P[j]$ is not the same as $T[i]$

- There are 3 possible cases.

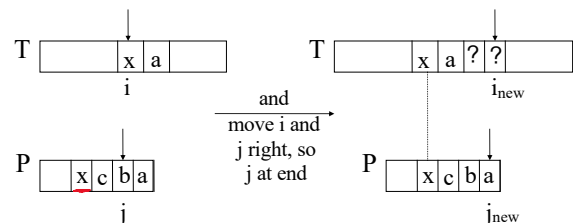


63

63

Case 1

- If P contains x somewhere, then try to *shift P* right to align the last occurrence of x in P with $T[i]$.

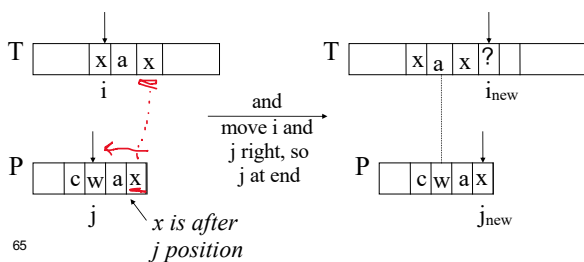


64

64

Case 2

- If P contains x somewhere, but a shift right to the last occurrence is *not* possible, then *shift P* right by 1 character to $T[i+1]$.

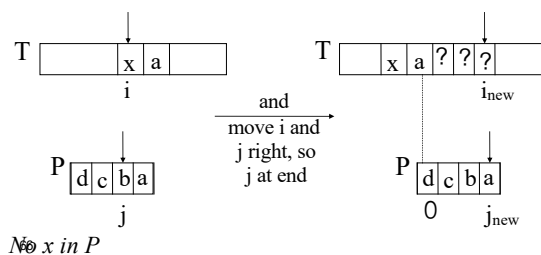


65

65

Case 3

- If cases 1 and 2 do not apply, then *shift P* to align $P[0]$ with $T[i+1]$.



No x in P

66

Boyer-Moore Example (1)

T:

a p a t t e r n m a t c h i n g a l g o r i t h m

r i t h m r i t h m r i t h m r i t h m

P:

r i t h m r i t h m r i t h m

67

67

Last Occurrence Function

- Boyer-Moore's algorithm preprocesses the pattern P and the alphabet A to build a last occurrence function L()
 - L() maps all the letters in A to integers
- L(x) is defined as: // x is a letter in A
 - the largest index i such that P[i] == x, or
 - 1 if no such index exists

68

68

L() Example

- A = {a, b, c, d}
- P: "abacab"

P a b a c a b
0 1 2 3 4 5

x	a	b	c	d
L(x)	4	5	3	-1

L() stores indexes into P[]

69

69

Boyer-Moore Example (2)

T: a b a c a a b a d c a b a c a b a a b b

P: a b a c a b

a b a c a b a b a c a b a b a c a b a b a c a b a b a c a b

x	a	b	c	d
L(x)	4	5	3	-1

70

70

Analysis

- Boyer-Moore worst case running time is $O(nm + A)$
- But, Boyer-Moore is fast when the alphabet (A) is large, slow when the alphabet is small.
 - e.g. good for English text, poor for binary
- Boyer-Moore is *significantly faster than brute force* for searching English text.

71

71

Worst Case Example

- T: "aaaaa...a"

T: a a a a a a a a a a

- P: "baaaaa"

P: b a a a a a a

b a a a a a a

b a a a a a a

b a a a a a a

72

72

Boyer-Moore Example (2)

T: a b a c a b a d c a b a c a b a a b b

P: a b a c a b

a b a c a b a b a c a b

a b a c a b a b a c a b

a b a c a b

x	a	b	c	d
L(x)	4	5	3	-1

73

73

Boyer-Moore: Good suffix rule

If t is the longest suffix of P that matches T in the current position, then P can be shifted so that the previous occurrence of t in P matches T . In fact, it can be required that the character before the previous occurrence of t be different from the character before the occurrence of t as a suffix. If no such previous occurrence of t exists, then the following cases apply:

- Find the smallest shift that matches a prefix of the pattern to a suffix of t in the text
- If there's no such match, shift the pattern by n (the length of P)

74

74

Boyer-Moore: Good suffix rule

- Consider the example in the paper:

• $P =$ ABCXXXABC

• YXABC ...

• ABCXXXABC

0 1 2 3 4 5 6 7 8

→

- -6 -5 -4 -3 -2 -1 -3 -2 7

75

75

Boyer-Moore: Good suffix rule

- Consider the example in the paper:

• $P =$ ABYXCDEYX

• AEYX ...

• ABYXCDEYX

0 1 2 3 4 5 6 7 8

←

- -9 -8 -7 -6 -5 -4 -1 -2 7

76

76

Boyer-Moore: Good suffix rule

- Consider the examples in the paper:

- ABCXXXABC
- ABYXCDEYX

- -6 -5 -4 -3 -2 -1 -3 -2 7
- -9 -8 -7 -6 -5 -4 1 -2 7

77

77

Boyer-Moore: Good suffix rule

- Another example:

• abacab

• abacab

0 1 2 3 4 5

- -4 -3 -2 -1 -2 4

x is the location where unmatched.
 $x=4$, means x points to pattern array's 4th.
 $x=-1$, means x points to pattern array's -1th.

78

78

Boyer-Moore Example (3)

T:

a	b	a	c	a	b	a	d	c	a	b	a	c	a	b	a	a	b	b
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

P:

a	b	a	c	a	b
---	---	---	---	---	---

Good suffix rule

x	a	b	c	d
$L(x)$	4	5	3	-1

79

KMP & BM

- Please refer to the original papers (available at WebCMS) for the details of the algorithms
- Most text processors use BM for “find” (& “replace”) due to its good performance for general text documents

80

80